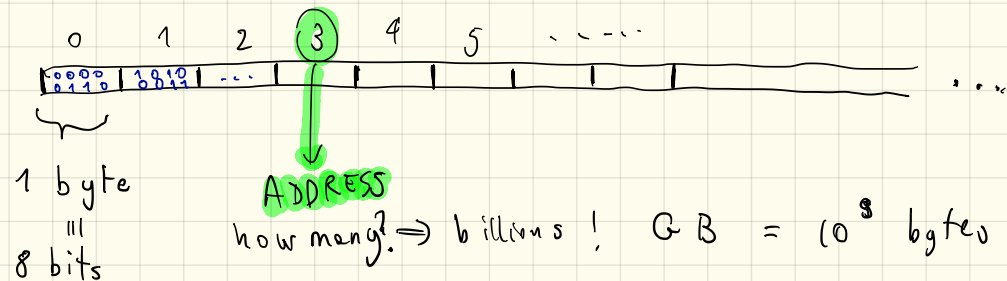


STORING ARRAYS IN MEMORY

The computer needs to write somewhere all the info that needs to be processed/stored

RAM $\rightarrow$ linear container



how many? $\Rightarrow$ billions! GB = 10^9 bytes

convention,
all the
computers
are nowadays
organized this
way

All the bits always have a
state 1/0, depending on
past computations, random initiali-
zation, ...

What is the most natural way of storing
a sequence of numbers in this memory?

E.G.: we want to store [2, 14, 3, 8]

Binary representation

2 → 0 0 0 0 0 0 1 0

14 → 0 0 0 0 1 1 1 0

3 → 0 0 0 0 0 0 1 1

8 → 0 0 0 0 1 0 0 0

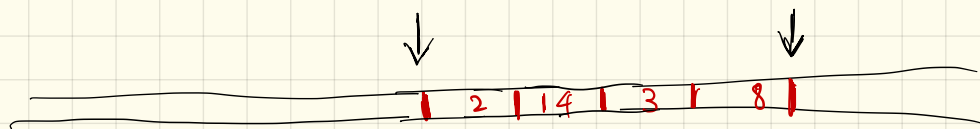
they all fit
in 1 byte!

For example

take 14 →

8 bits							
2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰
128	64	32	16	8	4	2	1
0	0	0	0	1	1	1	0

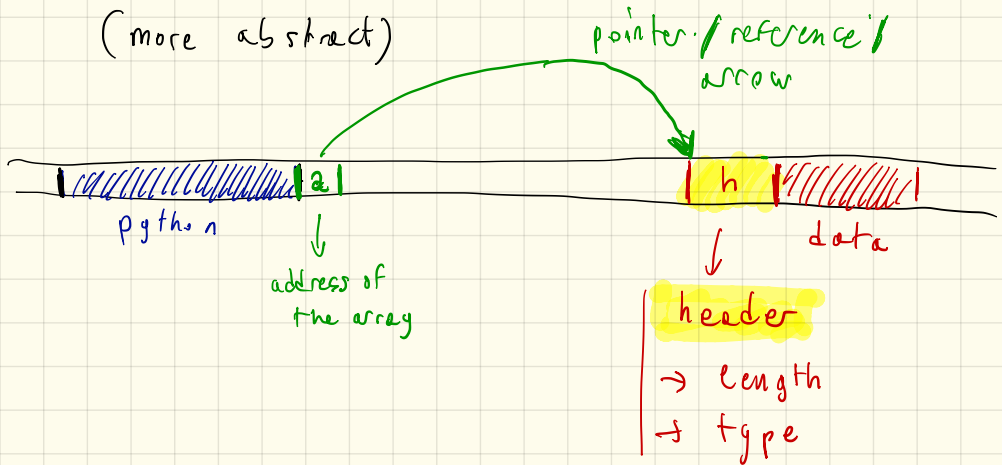
⇒ Use consecutive slots in the RAM :



start? end?

there is nothing special about these bits,
unless I have a flag or something that
points there!

Storing 1-d nd arrays (#1)



WHY DO I NEED THE TYPE?

→ $\text{uint } 8$ ⇒ each element takes 1 byte

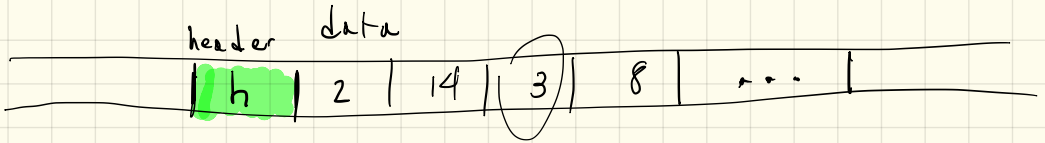
→ $\text{uint } 32$ ⇒ each element takes 4 bytes

⇒ homogeneity of the entries +
length +
type =

= now the CPU knows how to read the content!

Storing python list (#1)

Could I use the same compact representation for lists?



flexible → swap 3 for 3¹⁰⁰
ASSIGNMENT ↪ I need more bits to store

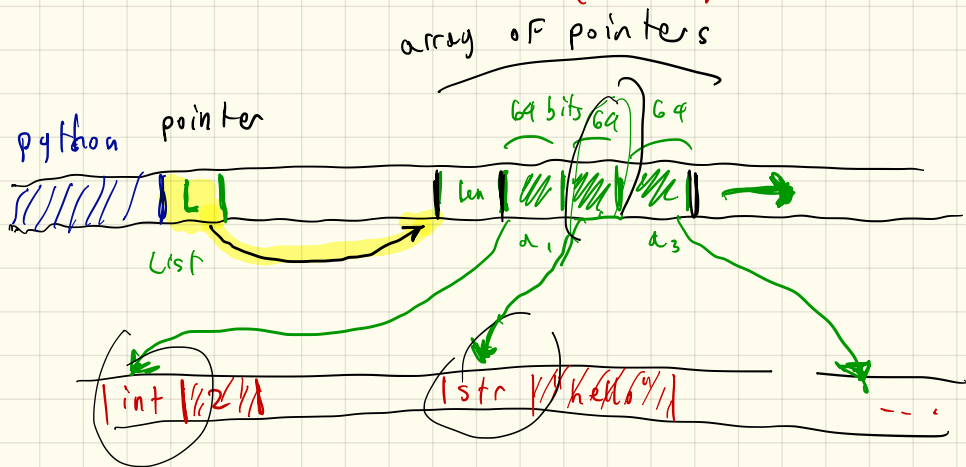
The operation would be $O(N)$, where N is the length of the vector ← it → I have to move all the rest of the entries

↪ we need $O(1)$
⇒ constant cost!

flexible TYPE → what if I swap 3 for "a", how does the program know how to interpret the bits?

IT DOES NOT WORK

Storing python list (#2)



NOW I NEED 2 JUMPS TO FETCH THE CONTENT OF THE LIST L, BUT

→ assignment becomes $O(1)$

- write new content somewhere else
- update the pointer

why is it efficient to do operations within same type?

	int64						
	1	+	2.5	=	3.5		same result
					11		
same operation?	1.0	+	2.5	=	3.5		
	float64						

First row requires a conversion (int → float)

NO

Storing 2-d nd arrays (#1)

$$\begin{bmatrix} 1 & 2 & 5 \\ 3 & 14 & 7 \end{bmatrix}$$

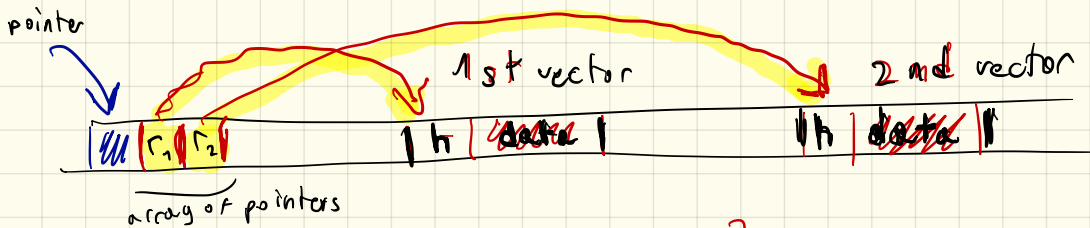
2D object :

how to store it ?

(RAM is 1D)

(NOT HOW NUMPY DOES IT!)

1st possible way $\rightarrow$ store each row as a separate vector, and then link them



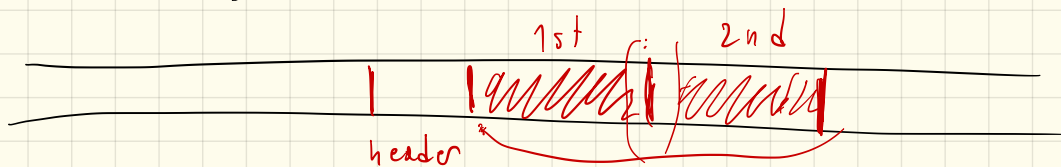
$[[1, 2, 5], [3, 14, 7]]$

this is indeed similar to how python would store a list of lists

add $\downarrow$ another layer of pointers

Storing 2-d and arrays (#1)

Numpy groups all the data in contiguous slots $\Rightarrow$ SINGLE HEADER



I NEED MORE
INFO NOW

type

size

= tot number of entries

dim

= 2

shape

= (n rows, n cols) =

= (2, 3)

In general d-dimensional tuple

In what order am I storing the table?

↑ Row-MAJOR ORDER $\rightarrow$ C, C++, python, ...
↓ transpose
↓ COLUMN-MAJOR ORDER $\rightarrow$ Fortran, Matlab, Julia, ...

Row-major $\Rightarrow$ go row by row!